TASK 20: BASICS OF WEB HOSTING AND DNS

OBJECTIVE

To understand the basics of web hosting and DNS, including web servers, DNS hierarchy, DNS records, DNS resolution, DNS caching, TTL, forward and reverse DNS, and practical DNS commands.

REQUIREMENTS

- Linux/RHEL system
- Internet connectivity
- Web browser
- nslookup
- dig
- host

PROCEDURE

STEP 1 – UNDERSTAND WEB HOSTING

Web hosting is the process of storing website files and applications on a server so that they can be accessed through the internet.

A web server such as Apache or NGINX receives requests from clients and provides the requested website content.

Client / Browser
↓
Internet
↓
Web Server
↓
Website Content

How Web Hosting Works

When a user enters a website address:

www.example.com

the overall process is:

User
↓
Browser
↓
DNS
↓

IP Address
↓
Web Server
↓
Website Files / Application
↓
Browser
Example

Suppose:

www.example.com
↓
DNS
↓
192.0.2.10

The browser then connects to:

192.0.2.10

The web server running on that machine receives the request and sends the required website content back to the browser.

Types of Web Hosting

1. Shared Hosting

Multiple websites share resources on the same server.

Server
├── Website A
├── Website B
├── Website C
└── Website D

It is commonly used for smaller websites.

2. VPS Hosting

A physical server is divided into multiple virtual servers.

Physical Server
├── VPS 1
├── VPS 2
└── VPS 3

Each VPS has its own allocated resources and operating environment.

3. Dedicated Hosting

A complete physical server is allocated to one customer or organization.

Physical Server
↓
One Customer

4. Cloud Hosting

Website or application resources run using cloud infrastructure.

Examples include cloud services provided by AWS, Microsoft Azure, and Google Cloud.

A typical cloud hosting architecture can look like:

Internet
↓
DNS
↓
Cloud Load Balancer
↓
Web/Application Server
↓
Database

2 – UNDERSTAND DNS

DNS stands for Domain Name System.

DNS translates human-readable domain names into IP addresses.

Example:

example.com
↓
DNS
↓
192.0.2.10

DNS allows users to access websites using domain names instead of remembering IP addresses.

STEP 3 – UNDERSTAND DNS HIERARCHY

DNS follows a hierarchical structure:

Root (.)
  ↓
TLD (.com / .org / .in)
  ↓
Domain (example.com)
  ↓
Host (www)

The major components of DNS hierarchy are:

- Root DNS servers
- TLD DNS servers
- Authoritative DNS servers
- Recursive DNS resolvers

STEP 4 – WORKING OF DNS

When a user enters a domain name such as:

www.example.com

the computer needs to find the IP address associated with that domain before connecting to the web server.

The DNS resolution process works as follows:

1. User enters the domain

The user enters:

https://www.example.com

in a web browser.

2. Browser checks DNS cache

The browser may first check whether it already has a valid DNS result stored in its cache.

If the required information is available, the cached result can be used.

3. Operating system checks its cache

If the browser does not have the required information, the operating system may check its local DNS cache.

4. Request is sent to the DNS resolver

If the IP address is not available locally, the computer sends the DNS query to a recursive DNS resolver.

Client
↓
DNS Resolver

5. Resolver checks its cache

The DNS resolver checks whether it already has a valid cached answer.

If the answer is available, it can return the IP address directly.

If the answer is not available, the resolver continues the lookup.

6. Resolver contacts the Root DNS

The resolver contacts the root DNS system to determine where information about the required Top-Level Domain can be found.

For:

www.example.com

the TLD is:

.com

7. Resolver contacts the TLD DNS server

The resolver contacts the .com TLD DNS server.

The TLD server provides information about the authoritative DNS servers responsible for example.com.

8. Resolver contacts the Authoritative DNS server

The resolver then queries the authoritative DNS server for the required DNS record.

For example:

www.example.com A 192.0.2.10

The authoritative DNS server returns the requested IP address.

9. Resolver returns the IP address

The resolver sends the result back to the user's computer.

Authoritative DNS
↓
DNS Resolver
↓
Client
↓
192.0.2.10

10. Browser connects to the web server

The browser can now use the IP address to connect to the web server.

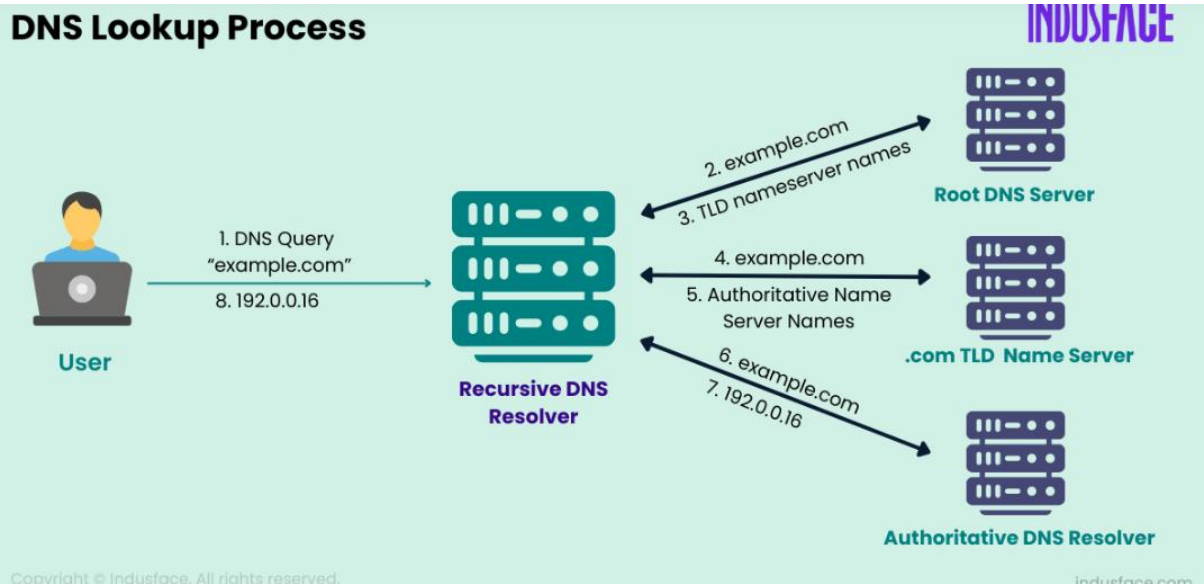
For HTTPS:

Browser
↓
192.0.2.10:443
↓
Web Server
↓
Website

COMPLETE DNS WORKING

User
↓
Browser
↓
Browser / OS Cache
↓
DNS Resolver
↓
Root DNS
↓
TLD DNS
↓
Authoritative DNS
↓
DNS Record
↓
IP Address
↓
Web Server
↓
Website

DNS is responsible for resolving a domain name to the required DNS information, such as an IP address. After obtaining the IP address, the browser can connect to the web server.



STEP 5 – UNDERSTAND DNS RECORDS

DNS records contain information about a domain.

1. A Record — Address Record

An **A record** maps a domain name to an **IPv4 address**.

Example:

example.com → 192.0.2.10

Purpose: Used to point a domain or hostname to an IPv4 server address.

2. AAAA Record

An **AAAA record** maps a domain name to an **IPv6 address**.

Example:

example.com → 2001:db8::10

Purpose: Used to point a domain or hostname to an IPv6 server address.

3. CNAME Record — Canonical Name

A **CNAME record** creates an alias from one domain name to another domain name.

Example:

www.example.com → example.com

Purpose: Used when one hostname should refer to another hostname.

4. MX Record — Mail Exchange

An **MX record** specifies the mail server responsible for receiving email for a domain.

Example:

example.com → mail.example.com

It also contains a priority value.

MX 10 mail.example.com

Purpose: Used for email delivery.

5. NS Record — Name Server

An **NS record** identifies the authoritative DNS servers for a domain.

Example:

example.com → ns1.example.com
example.com → ns2.example.com

Purpose: Specifies which DNS servers are authoritative for the domain.

6. PTR Record — Pointer

A **PTR record** is used for **reverse DNS lookup**.

It maps an IP address to a domain/hostname.

192.0.2.10 → server.example.com

Purpose: Used for reverse DNS.

7. SOA Record — Start of Authority

An **SOA record** contains important administrative information about a DNS zone.

It includes information such as:

- Primary authoritative DNS server
- Administrative contact
- Serial number
- Refresh interval
- Retry interval
- Expiration information
- Negative caching information

Purpose: Defines important authority and administrative information for a DNS zone.

8. SRV Record — Service Record

An **SRV record** specifies the location of a particular service.

It can provide:

- Service
- Protocol
- Priority
- Weight
- Port
- Target hostname

Example:

```
_service._tcp.example.com
      ↓
server.example.com
      ↓
Port 443
```

Purpose: Used for service discovery, allowing applications to find the server and port for a specific service.

9. TXT Record — Text Record

A **TXT record** stores text information associated with a domain.

Common uses include:

- Domain verification
- SPF information
- Email security configuration
- Service verification

Example:

```
example.com → "verification=value"
```

Purpose: Stores text-based information used by services and administrators.

10. CAA Record — Certification Authority Authorization

A **CAA record** specifies which Certificate Authorities (CAs) are authorized to issue TLS/SSL certificates for a domain.

Example:

example.com → CAA → allowed certificate authority

Purpose: Helps control which Certificate Authorities can issue certificates for the domain.

STEP 6 – UNDERSTAND DNS CACHING AND TTL

DNS resolvers can temporarily cache DNS responses.

For example:

example.com → 192.0.2.10

The resolver can store this response and use it for subsequent requests until the cache expires.

TTL stands for Time To Live.

TTL determines how long a DNS response can generally remain cached.

Example:

TTL = 3600 seconds
3600 seconds = 1 hour

DNS caching reduces repeated DNS queries and improves DNS response performance.

STEP 7 – FORWARD DNS

Forward DNS resolves a domain name into an IP address.

Domain → IP Address

Example:

example.com
↓
192.0.2.10

A forward DNS lookup can be performed using:

dig example.com

STEP 8 – REVERSE DNS

Reverse DNS resolves an IP address into a domain name.

IP Address → Domain

Example:

192.0.2.10
↓
server.example.com

Reverse DNS commonly uses a PTR record.

The lookup can be performed using:

dig -x 192.0.2.10

STEP 9 – DNS LOOKUP USING NSLOOKUP

A basic DNS lookup can be performed using:

nslookup google.com

To query an A record:

nslookup -type=A google.com

To query an MX record:

nslookup -type=MX google.com

To query an NS record:

nslookup -type=NS google.com

To query a TXT record:

nslookup -type=TXT google.com

STEP 10 – DNS LOOKUP USING DIG

A basic DNS query can be performed using:

dig google.com

A simplified result can be obtained using:

dig +short google.com

Specific DNS records can be queried using:

dig google.com A
dig google.com MX
dig google.com NS
dig google.com TXT

dig google.com SOA

STEP 11 – DNS LOOKUP USING HOST

A basic DNS lookup can be performed using:

```
host google.com
```

Specific record types can also be queried:

```
host -t MX google.com
host -t NS google.com
```

VERIFICATION

The following concepts and activities were studied:

- Web hosting and web server concepts
- DNS and its purpose
- DNS hierarchy
- Working of DNS
- DNS resolution process
- DNS records
- DNS caching
- TTL
- Forward DNS
- Reverse DNS
- nslookup command
- dig command
- host command

CONCLUSION

Through this task, the fundamentals of web hosting and DNS were studied. The DNS hierarchy and complete DNS resolution process were understood, including the roles of the DNS resolver, Root DNS, TLD DNS, and Authoritative DNS servers.

Important DNS records such as A, AAAA, CNAME, MX, NS, TXT, PTR, and SOA were studied. DNS caching and TTL were also understood.

Practical DNS commands such as nslookup, dig, and host were studied for performing DNS lookups and inspecting different DNS record types.

This task provided a basic understanding of how domain names are resolved into IP addresses and how DNS works together with web hosting to make websites accessible over the internet.